# Track-System – Glossary

Last updated: 17 Oct 2026 (after step 5) | The words that keep coming back in this project, each one in plain language | **[NEW]** = added in step 5

## 1. The big picture – who talks to whom

```
FRONTEND (in the browser)    BACKEND (on the computer)    DATABASE
React + Vite (port 5173)     ASP.NET Core (C#, port 5163) SQL Server
   |                               |                          |
   |--- axios sends HTTP req ---> endpoint /api/...          |
   |    (+ the token)              |--- EF Core: C# to SQL -->|
   |<------ JSON answer ----------|<--------- rows ----------|

The frontend NEVER talks to the database directly. Always through the backend.
[NEW] The browser lets 5173 call 5163 only because the backend allows it (CORS).
[NEW] dotnet test starts the same backend in memory and talks to a separate DB: FinalProject_Test.
```

## 2. The platform words

| Word | What it means |
|---|---|
| .NET | Microsoft's platform for running C# programs, like Node.js is for JavaScript. It includes the compiler, the libraries and the runtime. (dotnet --version) |
| ASP.NET Core | The part of .NET for building web servers and APIs. Our whole backend. |
| SDK / Runtime | SDK = the tools to build code. Runtime = only what is needed to run it. A developer needs the SDK. We installed the SDK. |
| Kestrel | The actual web server inside ASP.NET Core. It listens on the port and receives the requests. (The line "Now listening on: ...") |
| NuGet | The library store for .NET. Every PackageReference in the .csproj is downloaded from there. dotnet add package <name> downloads one and adds it to the .csproj. |
| **[NEW]** Transitive package | A library we did not add ourselves; another library needs it and brings it in. Example: EF Core SqlServer -> Microsoft.Data.SqlClient -> System.Security.Cryptography.Pkcs. |
| **[NEW]** Direct reference wins | If the .csproj names a package version directly, that version is used instead of the transitive one. That is how we replaced the vulnerable Pkcs 9.0.11 with 10.0.13. |
| **[NEW]** Vulnerability warning (NU1902 / NU1903) | A NuGet warning that a package version has a known security problem (moderate / high), with a GitHub advisory link. Fix: move to a patched version. |
| npm | The same idea for JavaScript. The frontend's package.json is its shopping list. (npm install) |
| **[NEW]** package-lock.json | npm's exact list of installed versions. An npm install can rewrite it; if no package changed, undo it with git restore. |
| Vite | The tool that runs the React project while developing and builds it for production. Usually port 5173. (npm run dev) |
| **[NEW]** .env.development / VITE_ prefix | The frontend's settings file. Only names that start with VITE_ reach the browser code, read as import.meta.env.VITE_API_URL. Not in git: each PC creates it. |
| Configuration | Everything .NET reads at startup, merged into one place: appsettings.json + User Secrets **[NEW]** + appsettings.{Environment}.json. The code reads it with builder.Configuration["Jwt:Issuer"]. |
| Section:Key ( : ) | The colon means "inside". Jwt:Key = the Key inside the Jwt section, whether it came from appsettings.json or from User Secrets. |
| Development environment | The mode set in launchSettings.json (ASPNETCORE_ENVIRONMENT). User Secrets are loaded only in Development. |
| **[NEW]** Environment | The name of the mode the server runs in: Development (dotnet run), Testing and RateLimitTesting (our tests). Each one may have its own settings file. Try: dotnet run -- --environment Testing. |
| **[NEW]** appsettings.{Environment}.json | Loaded on top of appsettings.json and overrides only the keys it contains. appsettings.Testing.json swaps the DB to FinalProject_Test and the limit to 1000. |

## 3. The web words

| Word | What it means |
|---|---|
| HTTP request | A message the browser sends to a server: a method (GET/POST/...), an address, optional headers and optional body data. |
| GET / POST / PUT / DELETE | GET = give me data. POST = here is new data (for example login details). PUT = update something. DELETE = remove it. |
| Endpoint | One address on the server plus the code that answers it. Example: /api/health. |
| API | The whole collection of endpoints: the menu of everything the backend can do for the frontend. |
| REST | A common style for designing an API: addresses are nouns (/api/employees), methods are verbs. |
| JSON | The text format both sides speak, for example {"status":"ok"}. |
| Status code | A number describing the result: 200 OK, 201 created, 400 bad request, 401 not logged in, 403 not allowed, 404 address not found, 500 server crashed. |
| **[NEW]** 400 Bad Request (ours) | Wrong input. change-password answers 400 + a message (wrong current password, same password, broken rule). React shows the message and keeps the user logged in. |
| **[NEW]** 401 vs 403 (ours) | 401 = no valid token (React deletes it and goes to Login). 403 = the token is fine but not allowed here, e.g. mustChangePassword "true" on /api/employees. |
| 405 Method Not Allowed | The address exists but not for this method. Opening a POST-only endpoint in the browser (which sends GET) gives a blank page with 405. |
| 423 Locked | Our answer for a locked account in step 4. **[NEW]** No longer used: a locked account now gets the same generic 401, so the lock stays silent. |
| 429 Too Many Requests | The answer when one IP sends too many attempts. **[NEW]** Now real: login and change-password, 10 per minute per IP, with a JSON message. |
| Header | A name: value line sent with a request or answer, outside the body. Example: Content-Type: application/json. |
| Authorization header | The standard header for credentials. We send: Authorization: Bearer <token>. |
| Bearer | "The bearer (holder) of this token". Whoever presents a valid token is treated as its owner, which is why a stolen token is dangerous until it expires. |
| WWW-Authenticate | A header the server adds to a 401 explaining why the token was refused, e.g. error="invalid_token", "The signature is invalid" or "The token expired". |
| Request body | The data part of a POST, for example {"eNum":1,"password":"..."}. |
| IP address / ::1 | The caller's network address, saved in LoginLogs. ::1 is localhost in IPv6 (the same as 127.0.0.1). **[NEW]** In tests it is NULL: the in-memory server has no network. |
| localhost | "This computer". Not reachable from outside, which is what we want while developing. |
| Port | The door number on the machine. 5163 = our API, 5173 = React. |
| axios | A JavaScript library that sends HTTP requests from React. **[NEW]** Our one shared copy is in src/api/client.js. |
| **[NEW]** baseURL | The address axios puts in front of every call, so pages only write "/api/...". Ours comes from VITE_API_URL. |
| **[NEW]** Interceptor | An axios hook that runs on every request or answer. Ours: before each request add the Bearer token; after a 401 (not from login) remove the token and go to "/". |
| **[NEW]** sessionStorage | Browser storage for one tab. The token lives there: it survives F5 and is gone when the tab closes. |
| CORS | A browser security rule: a page from port 5173 may not call port 5163 unless the server allows it. **[NEW]** Done: policy "ReactApp" allows only the origins in Cors:AllowedOrigins. |
| **[NEW]** Origin | Scheme + host + port of the page, e.g. http://localhost:5173. CORS rules are per origin. |
| Connection refused | The browser's "can't reach this page": nothing is listening on that address. The server is not running or the port is wrong. |
| camelCase JSON | ASP.NET Core writes C# names (FirstName) into JSON as firstName automatically. |

| Word | What it means |
|---|---|
| Rate limiting | Limiting how many requests one IP may send in a time window. **[NEW]** Built into .NET: AddRateLimiter + UseRateLimiter, our policy is called "auth". |
| **[NEW]** favicon.ico 404 | Browsers ask every site for a tab icon. Our API has none, so this red line in the Console is harmless. |

## 4. The database words

| Word | What it means |
|---|---|
| SQL Server | Microsoft's database program. SQL Express is the free local edition. |
| SSMS | SQL Server Management Studio, the app where you look at the tables by hand. F5 runs everything in the current tab; highlight lines first to run only them. |
| Connection string | One line saying which server, which database and how to log in. In appsettings.json. |
| Windows Authentication | Logging into the DB with your Windows user, so no password is written anywhere. |
| ORM | Object Relational Mapper: maps database rows to objects in code. |
| EF Core | Entity Framework Core, Microsoft's ORM. Translates our C# into SQL and rows back into objects. |
| DbContext | The EF Core class that represents the database. Every query goes through our AppDbContext. |
| DbSet | One line inside the DbContext representing one table. Ours: Employees and LoginLogs. |
| Model / Entity | A plain C# class whose properties mirror the columns of one table (Employee.cs, LoginLog.cs). EF Core fills it. |
| Migration | EF Core creating tables from C# classes. We do NOT use it: our tables already exist. |
| Provider | The package that knows one database's dialect. Ours: Microsoft.EntityFrameworkCore.SqlServer. |
| Primary key | The column that identifies each row uniquely (Employees.eNum, LoginLogs.logID). |
| IDENTITY | The DB fills the number automatically (1, 2, 3...). Numbers are never reused, even after a DELETE. **[NEW]** A dropped and recreated DB starts again at 1. |
| NULL / NOT NULL | NULL = the column may be empty. NOT NULL = a value is required. |
| DEFAULT | A value the DB fills in when none is given, for example role = 'Employee'. |
| Foreign key | A rule that a column must point to an existing row in another table (Employees.teamNum -> Teams). A JOIN does not need it; it only enforces the link. |
| Log table without FK | LoginLogs.eNum has no foreign key on purpose: it stores the number that was typed, so attempts by unknown numbers are logged too and history survives a deletion. |
| sys.foreign_keys | A system view listing every foreign key, used to find an auto-generated FK name. |
| DROP CONSTRAINT | ALTER TABLE ... DROP CONSTRAINT [name] removes a rule (like an FK) without touching the data. |
| UPDATE ... SET ... WHERE | Changes existing rows. Always with WHERE, or every row changes. |
| UTC | World time with no time zone and no daylight saving. Israel is UTC+3 in summer, UTC+2 in winter. lockoutEnd is stored in UTC. |
| GETDATE() / GETUTCDATE() | SQL Server's local time / UTC time. Compare lockoutEnd with GETUTCDATE(). |
| AT TIME ZONE | Converts a time for display: lockoutEnd AT TIME ZONE 'UTC' AT TIME ZONE 'Israel Standard Time'. |
| Empty result [] | The query worked but the table has no rows. |
| **[NEW]** Test database (FinalProject_Test) | A separate DB only for dotnet test, on the same server. It holds only Employees and LoginLogs, is dropped and rebuilt on every run, and may be deleted any time. |
| **[NEW]** EnsureDeleted / EnsureCreated | EF Core methods: drop the whole database named in the connection string / create it with tables built from the DbSets. Used only for the test DB, behind a name check. |

## 5. The C# and code words

| Word | What it means |
|---|---|
| Namespace | The address of a class so other files can find it with using .... Ours: TrackSystem.Api.Data, .Models, .Dtos, **[NEW]** .Security, and TrackSystem.Api.Tests.*. |
| using | A line at the top that lets the file see the classes of a namespace, e.g. using System.Security.Claims. |
| Class / Property | A class is a blueprint. Its properties are the fields inside it. |
| Inheritance ( : ) | class AppDbContext : DbContext receives all of DbContext's abilities. **[NEW]** Also TestApiFactory : WebApplicationFactory<Program>. |
| Lambda ( => ) | A small function written inline and run later, e.g. the code inside MapGet. |
| async / await | Waiting for the DB without freezing the server. |
| **[NEW]** Task / async Task | The return type of an async method that gives back nothing. Our tests are async Task, so xUnit waits for them. Task<int> = gives back an int later. |
| Dependency Injection | You ask for something in the parentheses and the server hands you a ready one (AppDbContext db, HttpContext http, **[NEW]** ClaimsPrincipal user). |
| Service registration | Teaching the server how to build those objects (builder.Services...), before Build(). |
| **[NEW]** Scope (CreateScope) | A short life for scoped services like AppDbContext. Each request gets one; test helpers create one with Services.CreateScope(). |
| Middleware | Code every request passes through before the endpoint. Ours, in order: **[NEW]** UseCors, UseRateLimiter, then UseAuthentication, UseAuthorization. |
| Minimal API | Endpoints written directly in Program.cs (the older style is Controllers). |
| Attribute [ ] | A label in square brackets about the ONE thing right after it. Example: [Key]. |
| [Key] | Tells EF Core which property is the primary key (ENum, LogID). |
| ENum vs enum | ENum is our property name; enum is a C# keyword. C# is case-sensitive, so they never clash. |
| Nullable ? | int? or string? means "may be empty": the C# side of a NULL column. **[NEW]** Also for parameters: string? token may be null (= send no header). |
| = string.Empty | A starting value of "" so a required text property is never null. |
| Types mapping | nvarchar -> string, int -> int, bit -> bool, datetime -> DateTime. |
| LINQ | C#'s query methods (Select, Where...). EF Core turns them into SQL. |
| .Select(...) | Chooses which fields to take, which is how the password never leaves the DB. |
| Anonymous object | new { ... } with no class name, used to shape the JSON output **[NEW]** and the JSON bodies in tests. |
| ToListAsync() | Runs the query and returns the rows as a list. |
| FirstOrDefaultAsync(...) | Returns the first matching row, or null if none. Used to find the employee by eNum. |
| Results.Ok(...) | Returns HTTP 200 with the object as JSON. |
| Results.Json(obj, statusCode) | Returns any status code with a JSON body. Used for the generic 401 with a message (**[NEW]** 423 is no longer used). |
| **[NEW]** Results.BadRequest / Results.Unauthorized | 400 with a JSON message / an empty 401. Used in change-password and refresh. |
| Results.Text(...) | Returns plain text (used once, in the temporary hash endpoint). |
| StatusCodes.Status401Unauthorized | Built-in named constants for the status numbers (401, **[NEW]** 429...). |
| DTO | Data Transfer Object: a class describing exactly what goes in or out of an endpoint. Ours: LoginRequest, **[NEW]** ChangePasswordRequest. |
| record | A short way to write a class that only holds data: public record LoginRequest(int ENum, string Password); |
| **[NEW]** static class | A class you never create with new; you call its methods directly: PasswordRules.Validate(...). |
| Change tracking | EF Core remembers the original values of loaded rows. Changing a property marks the row as changed. |

| Word | What it means |
|---|---|
| Add(...) | db.LoginLogs.Add(row) puts a new row on EF's "to save" list. Nothing is written yet. |
| SaveChangesAsync() | Writes everything on the "to save" list to SQL Server in one trip (UPDATEs and INSERTs together). |
| Local function | A small function written inside another one. AddLog(...) inside the login endpoint, **[NEW]** CreateToken(employee) in Program.cs. |
| HttpContext | The object describing the current request: headers, caller's IP (http.Connection.RemoteIpAddress), user. |
| **[NEW]** ClaimsPrincipal user | The caller as the token guard saw them. user.FindFirst("sub")?.Value gives the eNum as text. |
| **[NEW]** int.TryParse(text, out var n) | Tries to turn text into a number: true + the number, or false (no crash). Used on the sub claim. |
| **[NEW]** StringComparison.OrdinalIgnoreCase | Compares text ignoring capitals: "Active" equals "active". Used for Status. |
| ?? (null-coalescing) | "If the left side is null, use the right side." Used to stop the server when Jwt:Key is missing. |
| ?. (null-conditional) | "Only continue if not null." RemoteIpAddress?.ToString() gives null instead of crashing. |
| DateTime.UtcNow | The current time in UTC. Used for lockoutEnd and token expiry. |
| TimeZoneInfo | Converts UTC to a time zone. "Asia/Jerusalem" gives Israel time for LoginLogs. |
| GetValue<int>(...) | Reads a setting and converts it to a number (ExpiresMinutes). **[NEW]** GetValue("RateLimit:AuthPermitLimit", 10) also gives a fallback if the key is missing. |
| Method chaining | }) followed by .RequireAuthorization(); calls another method on the result, so the statement ends only after the last call. |
| **[NEW]** Default / named parameters | CreateEmployeeAsync(string password, string status = "active", ...): skipped ones keep their default. status: "pending" sets just that one. |
| **[NEW]** const | A value fixed when compiling, e.g. const string Password. Only constants may go inside attributes like [InlineData(Password, ...)]. |

## 6. The security words

| Word | What it means |
|---|---|
| Hashing | Turning a password into scrambled text that cannot be turned back. We store and compare hashes only. |
| BCrypt | The hashing library we use. Deliberately slow, which makes guessing attacks expensive. |
| HashPassword / Verify | BCrypt.HashPassword(pwd) makes a 60-char hash. BCrypt.Verify(typed, hash) returns true/false by re-hashing the typed password with the salt stored in the hash. |
| $2a$11$... | The hash format: version ($2a$), work factor (11 = 2^11 rounds), 22 chars of salt, then the hash. |
| Work factor (cost) | How many rounds BCrypt runs. Higher = slower = harder to crack. |
| Salt | Random characters added before hashing, so the same password gives different hashes. |
| Lockout | After 5 wrong passwords the account is locked for 15 minutes (failedLoginAttempts, lockoutEnd). A correct login resets the counter. **[NEW]** Silent: while locked, even the correct password gets the generic 401. |
| Generic error message | The same 401 text for a wrong number and a wrong password, so nobody learns which employee numbers exist (user enumeration). **[NEW]** Also for locked and not-active accounts. |
| Timing attack | Learning something from how long an answer takes. **[NEW]** Fixed: unknown eNums and locked accounts now run the same slow BCrypt check. |
| **[NEW]** Dummy hash | A real BCrypt hash of a throwaway text, made once at startup. Checked (result ignored) for unknown eNums and locked accounts, so every failed login takes about the same time. |
| **[NEW]** Active-only login | A correct password still gets the generic 401 if Status is not "active" (pending / inactive). The wrong-password counter is not touched. |

| Word | What it means |
|---|---|
| [NEW] Password rules | A new password needs 8-64 characters, a capital, a number and a special character (not a letter, digit or space). 64 because BCrypt ignores everything after 72 bytes. |
| Audit log | A record of events for later checks. LoginLogs: every attempt, with Israel time, IP and result. |
| Authentication | Who are you? The login, and the token check on every request. |
| Authorization | What are you allowed to do? The rules: which endpoint needs a token or a role. |
| JWT | JSON Web Token: a signed string the server gives after a successful login. React sends it back on every later request. |
| Header . Payload . Signature | The 3 parts of a JWT, separated by dots. Header = algorithm (HS256). Payload = the claims (readable by anyone). Signature = proof it was made with our key and not changed. |
| [NEW] Tampered token | A token whose payload was edited (e.g. role Employee -> Admin) with the old signature kept. The server recomputes the signature, it does not match -> 401. Proven by a test. |
| Claim | One piece of data inside the token: a name and a text value. |
| sub | Subject: who the token belongs to. Ours: the eNum, e.g. "1". |
| role | What the user is (Employee / Manager / Admin). Used later for role rules. |
| mustChangePassword | Our own claim: "true" on first login. [NEW] Enforced: such a token opens only change-password and refresh (403 elsewhere). |
| iss / aud | Issuer = who made the token (TrackSystem.Api). Audience = who it is for (TrackSystem.Client). |
| exp / iat / nbf | Expires at / issued at / not valid before, as seconds since 1/1/1970 (UTC). |
| Token expiry | [NEW] Our tokens live 30 minutes (Jwt:ExpiresMinutes). After that: 401 and log in again, unless refresh renewed it. |
| [NEW] Refresh endpoint / sliding session | While the user is active, session.js asks POST /api/auth/refresh for a new 30-minute token (at most once a minute). 30 minutes without activity -> logged out. |
| [NEW] Idle logout | session.js checks every 15 s: token expired -> removed, back to Login, even without a click. |
| Signing key | The secret (64 random bytes, in User Secrets) that signs and checks tokens. Whoever has it can forge tokens. [NEW] Tests make their own random key on every run. |
| Symmetric key / HMAC-SHA256 (HS256) | Symmetric = the same key signs and checks. HS256 = the signing method: a SHA-256 hash mixed with the key. |
| SecurityTokenDescriptor | The "recipe" for one token: claims, issuer, audience, expiry, how to sign. |
| JsonWebTokenHandler | The "printer" that turns the recipe into the header.payload.signature text. [NEW] ReadJsonWebToken does the reverse (decode only, no signature check). |
| Stateless | The server keeps no list of tokens. Each request carries its token and is checked on the spot. Logout = React deletes its copy. [NEW] So an old token stays valid until it expires. |
| AddAuthentication / AddJwtBearer | Registers the token guard (the JwtBearer handler) and how it checks tokens. |
| TokenValidationParameters | The guard's checklist: signature, issuer, audience, lifetime, plus which claims are the name (sub) and the role. |
| ClockSkew | Allowed clock difference when checking expiry. Default 5 minutes; ours is 0, so [NEW] 30 minutes means exactly 30. |
| MapInboundClaims = false | Stops .NET from renaming sub and role to old long URL names (a compatibility habit from older Microsoft login systems). |
| UseAuthentication | Middleware: reads the Bearer token on every request and identifies the caller. Blocks nothing by itself. |
| AddAuthorization / UseAuthorization | Registers and runs the rules system. Blocks only endpoints that have a rule. |
| [NEW] Authorization policy | A named rule. Ours: the default policy (valid token AND mustChangePassword "false") and "PasswordChange" (any valid token). |
| [NEW] RequireClaim | Part of a policy: the token must hold this claim with this value, else 403. |

| Word | What it means |
|---|---|
| RequireAuthorization() | The rule on one endpoint. **[NEW]** Empty = the default policy; with a name = that policy, e.g. RequireAuthorization("PasswordChange"). |
| **[NEW]** Fixed window / partition key | Our limiter counts requests per 1-minute window. Each IP (the partition key) has its own counter; no IP -> one shared "unknown" counter (as in tests). |
| **[NEW]** RequireRateLimiting("auth") | Puts the "auth" limit on one endpoint: login and change-password share it. |
| User Secrets | Secret settings stored on your own PC, outside the project folder (AppData\Roaming\Microsoft\UserSecrets\<id>\secrets.json). |
| UserSecretsId | The name of the project's secrets folder, written in the .csproj. Not secret. |
| Never expose the hash | Even a hashed password is never sent to the browser. |
| jwt.io | A website that decodes a token so you can read its payload. Paste test tokens only, never the key. |

## 7. Tools and workflow

| Word | What it means |
|---|---|
| git add / commit / push | add = choose the changes. commit = save a snapshot with a message. push = upload to GitHub. |
| git status | Shows what changed before you commit. Check it every time: no secrets, no *.http files. |
| **[NEW]** git add -n <path> | Dry run: lists what git WOULD add, adds nothing. Good for checking a new folder. |
| **[NEW]** git diff --stat <file> | How many lines changed in a file, without showing them. |
| **[NEW]** git restore <file> | Throws away local changes to one file and puts back the committed version. |
| **[NEW]** git log -1 --oneline | Shows the last commit in one line. |
| .gitignore | The list of files git must ignore: bin, obj, node_modules, secrets, *.http **[NEW]** and .env.* (so .env.development too). |
| bin / obj | Folders .NET generates while building. Never edit them, never commit them. |
| Build vs Run | Build = compile. Run = start. dotnet run does both; dotnet build only checks it compiles. |
| Hot reload | The backend does not reload by itself: after every change, Ctrl+C and dotnet run again. |
| Working directory | The folder the terminal is in. API commands run in Backend\TrackSystem.Api, **[NEW]** test commands in Backend\TrackSystem.Api.Tests. |
| cd | Change directory, e.g. cd Backend\TrackSystem.Api. cd ..\.. goes up two folders. |
| MSB1003 | dotnet found no .csproj in the current folder. Fix: cd into the project folder. |
| Script Table as | SSMS menu that writes the SQL of an existing table. Do not press F5 in that tab: it tries to create the table again ("There is already an object named..."). |
| dotnet user-secrets init / set / list | init = create the project's secrets folder (once). set "Jwt:Key" "..." = save a secret. list = show them. |
| **[NEW]** dotnet new xunit | Creates a test project from the xUnit template (-n name, -o folder). |
| **[NEW]** dotnet add reference | Links one project to another, e.g. the tests to TrackSystem.Api. |
| **[NEW]** dotnet test | Builds and runs every test, then prints total / failed / succeeded. --logger "console;verbosity=detailed" lists each test. |
| **[NEW]** dotnet nuget why <project> <package> | Shows which package brought in a transitive package. |
| **[NEW]** dotnet list package --vulnerable --include-transitive | Lists packages with known security problems, including indirect ones. |
| **[NEW]** dotnet run -- --environment Testing | Starts the API in another environment (the args after -- go to our program). |

| Word | What it means |
|---|---|
| REST Client / .http file | A VS Code extension for sending requests from a text file. Click Send Request (not the URL, which opens the browser with GET). ### starts a new request. |
| # @name / {{login.response.body.token}} | REST Client variables: name a request, then reuse a field from its answer, like the token. |
| F12 -> Network | The browser's developer tools tab that shows each request and its status code. **[NEW]** Application -> Session Storage shows the saved token. |
| **[NEW]** F12 -> Console context "top" | Console commands run in the page selected at the top. It must be the localhost page, not another frame. |
| **[NEW]** npm install / "vite is not recognized" | npm install downloads the frontend libraries into node_modules. Without it, npm run dev cannot find Vite. |

## 8. The testing words

| Word | What it means |
|---|---|
| **[NEW]** Automatic test | Code that checks other code and answers pass / fail. dotnet test runs all of them in seconds, every time, the same way. |
| **[NEW]** Unit test | Tests one small piece alone, with no server and no DB. Ours: PasswordRulesTests. |
| **[NEW]** Integration test | Tests the pieces together: real HTTP request -> Program.cs -> EF Core -> SQL Server. Ours: everything in the Api/ folder. |
| **[NEW]** xUnit | The test framework we use. It finds the test methods, runs them and reports the result. |
| **[NEW]** Test project | A separate project (TrackSystem.Api.Tests) that holds only tests and references the API project. It is never part of the real server. |
| **[NEW]** [Fact] | Marks a method as one test with fixed data. |
| **[NEW]** [Theory] + [InlineData] | One test method run once per InlineData row; the row values arrive as the method's parameters. Each row counts as a test. |
| **[NEW]** Assert | The checks. Equal(expected, actual), Null / NotNull, True / False, Single (exactly one item, returns it), All (every item), InRange, Contains / DoesNotContain, NotEqual. A failed Assert stops the test. |
| **[NEW]** Expected vs actual | Assert.Equal(expected, actual): expected first. On failure xUnit prints both values. |
| **[NEW]** WebApplicationFactory<Program> | Microsoft's factory (package Microsoft.AspNetCore.Mvc.Testing) that starts our real Program.cs in memory, with no port, and gives HttpClients that call it. |
| **[NEW]** Factory | An object whose job is to create other things. TestApiFactory creates the test API, the clients, test employees and tokens. |
| **[NEW]** ConfigureWebHost / UseEnvironment / UseSetting | Where the factory changes the API before it starts: the environment ("Testing") and single settings (the random Jwt:Key). |
| **[NEW]** CreateClient() | Gives an HttpClient whose requests go straight into the in-memory API. |
| **[NEW]** Fixture | An object xUnit creates once and shares with many tests, e.g. our factory. |
| **[NEW]** IClassFixture<T> | One T shared by all tests of one class. |
| **[NEW]** Collection / ICollectionFixture<T> | Classes marked [Collection("Api")] share ONE T and run one after another, never in parallel. Needed because they share one test DB. |
| **[NEW]** IAsyncLifetime | Gives a fixture InitializeAsync (run once before its tests) and DisposeAsync (after). Our factory builds the test DB in InitializeAsync. |
| **[NEW]** Safety guard | A check that stops the tests before doing damage: TestApiFactory refuses to drop any DB that is not FinalProject_Test. |
| **[NEW]** Test isolation | Each test creates its own employee (random email / user name), so tests never depend on each other's data or order. |
| **[NEW]** Control test | A test that should pass, next to one that should fail, to prove the failure has only one cause. Our own-key token -> 200 next to the other-key token -> 401. |

| Word | What it means |
| --- | --- |
| **[NEW]** EnsureSuccessStatusCode() | Throws if the answer is not 2xx, so a helper like LoginAsync fails the test right away with the status. |
| **[NEW]** Base64UrlEncoder | Decodes / encodes token parts (URL-safe base64). Used to build the tampered token. |
| **[NEW]** ITestOutputHelper | xUnit's way to print text from a test into the report (_output.WriteLine). |